LECTURE 9

Transformers & Attention

Instructor Manual

Embeddings, positional encoding, queries, keys and values, multi-head attention — and the architecture of *Attention Is All You Need*

AK DHAMIJA, Researcher & Instructor, AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Unit 4: Simulation Strategies in Warfare — AI & Machine Learning in Warfare

This manual contains: the 45-minute session plan with timings; a slide-by-slide teaching guide for all 49 slides; key messages, facilitation tips, anticipated questions with model answers, misconceptions and in-class activities; live demonstrations with the interactive studio; consistency notes on the source material; the complete quiz answer key with rationale and scoring rules; and the assignment with marking scheme and full model solutions.

Contents

1 Lecture at a Glance **2**

2 Session Plan (45 Minutes) **3**

3 Slide-by-Slide Teaching Guide **3**

3.1 1. Overview (slides 2–3, about 1 min) . . . 4
3.2 2. Why Transformers (slides 4–7, about 2.5 min) . . . 4
3.3 3. Tokens & Embeddings (slides 8–12, about 3 min) . . . 4
3.4 4. Positional Encoding (slides 13–18, about 4 min) . . . 4
3.5 5. Query, Key, Value (slides 19–21, about 2 min) . . . 4
3.6 6. Worked Example (slides 22–28, about 4.5 min) . . . 5
3.7 7. Multi-Head Attention (slides 29–32, about 2.5 min) . . . 5
3.8 8. The Paper's Architecture (slides 33–43, about 7.5 min) . . . 5
3.9 9. Generation & Training (slides 44–45, about 1 min) . . . 6
3.10 10. Defence Applications (slides 46–47, about 1 min) . . . 6
3.11 11. Summary (slides 48–49, about 1 min) . . . 6

4 Key Messages **6**

5 Facilitation Tips **7**

6 Anticipated Questions and Model Answers **7**

7 Common Misconceptions **7**

8 In-Class Activities and Discussion Prompts **8**

9 Run It Live: Studio Demonstrations **8**

10 Quiz Administration and Answer Key **9**

11 Assignment: Marking Scheme and Model Solutions **12**

11.1 Problem 1: Sinusoidal positional encoding (20 marks) . . . 12
11.2 Problem 2: Scaled dot-product attention by hand (20 marks) . . . 13
11.3 Problem 3: Multi-head shapes and parameter counts (20 marks) . . . 13
11.4 Problem 4: Masking and cost (20 marks) . . . 14
11.5 Problem 5: BERT, GPT or encoder–decoder for a defence task (20 marks) . . . 14
11.6 Problem 6 (bonus): Add & Norm by hand (20 marks) . . . 15

1 Lecture at a Glance

Lecture	9 — Transformers & Attention
Duration	45 minutes: 5 min opening Q&A, 30 min teaching, 10 min auto-quiz
Slides	49 (Beamer PDF, 16:9), 11 sections
Prerequisite	Lecture 8; students should have attempted its quiz and assignment.
Student materials	`Lecture9_Slides_Beamer.pdf`; `Lecture9_Reference_Text.pdf`; `Lecture9_Quiz.html`; `Lecture9_Answers_Assignment.html` (release after the quiz)
Interactive studio	`Lecture9_Attention_Workbench.html` (live demonstrations in Section 9)
Assessment	Auto-quiz: 15 MCQs, 40 s each, $+1/-0.25$. Assignment: 5 problems, 100 marks, plus a 20-mark bonus problem.

Overview

Lecture 9 gives the technical treatment of Transformers, following Vaswani et al. (2017). Part 1 explains why recurrence was dropped (Table 1 of the paper), works word-vector arithmetic (tiger − man + woman → tigress, cosine 0.991), derives the sinusoidal positional encoding with a $d_{\text{model}} = 4$ example and the rotation argument for relative position, and computes scaled dot-product attention by hand on "enemy tank advances" (3 tokens, $d_k = 2$), including the causal mask and the variance argument for $\sqrt{d_k}$. Part 2 adds a second head, concatenation and Add & Norm on the same sentence, then walks every block of Figure 1 of the paper with the base model's shapes, the three uses of attention, the paper's settings and BLEU results, the three model families, next-token generation and training, and defence applications with their limits. It is the densest deck in the course and may be split over two sessions.

Learning outcomes

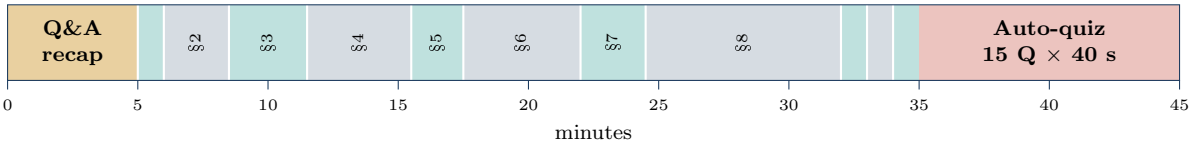
By the end of the lecture students should be able to:

1. Explain why self-attention replaced recurrence, using the complexity, sequential-operations and path-length comparison.
2. Compute word-vector arithmetic with cosine similarity and the sinusoidal positional encoding, and explain the rotation property.
3. Compute scaled dot-product attention by hand, with and without a causal mask, and justify the $\sqrt{d_k}$ scaling.
4. Run a two-head example with concatenation and Add & Norm, and count multi-head parameters.
5. Explain every block of Figure 1 of Vaswani et al., distinguish BERT, GPT and encoder–decoder models, and assess defence uses.

Syllabus mapping

- Deep learning for language and sequences — **Transformers and attention** (technical treatment)

2 Session Plan (45 Minutes)



Clock	Slides	Segment	Min
0:00–5:00	—	**Opening Q&A and recap** (see prompts below)	5
5:00–6:00	2–3	1. Overview (2 slides)	1
6:00–8:30	4–7	2. Why Transformers (4 slides)	2.5
8:30–11:30	8–12	3. Tokens & Embeddings (5 slides)	3
11:30–15:30	13–18	4. Positional Encoding (6 slides)	4
15:30–17:30	19–21	5. Query, Key, Value (3 slides)	2
17:30–22:00	22–28	6. Worked Example (7 slides)	4.5
22:00–24:30	29–32	7. Multi-Head Attention (4 slides)	2.5
24:30–32:00	33–43	8. The Paper's Architecture (11 slides)	7.5
32:00–33:00	44–45	9. Generation & Training (2 slides)	1
33:00–34:00	46–47	10. Defence Applications (2 slides)	1
34:00–35:00	48–49	11. Summary (2 slides)	1
35:00–45:00	—	**Auto-quiz** (instructions 1 min, 15 questions × 40 s = 10 min of answering; collect scores)	10

Slide 1 is the title slide and is shown while students settle. The teaching segment covers 48 content slides in 30 minutes, an average pace of 1.6 slides per minute.

Pitfall — pace

This pace is only achievable if most survey slides get one sentence each. See the facilitation tips for a fast path through a single session and a two-session alternative.

Opening Q&A prompts (first 5 minutes)

1. From Lecture 8: in the radio-net analogy, what are the query, the key and the value?
2. Why is attention order-blind, and what is added to fix it?
3. What does the causal mask stop the decoder from doing?

Take two or three answers per prompt; correct misconceptions briefly and note any topic that needs revisiting.

3 Slide-by-Slide Teaching Guide

Each subsection gives the slide range, the suggested time, what to emphasise, and the slide titles in order. Numbers refer to pages of the Beamer PDF.

3.1 1. Overview (slides 2–3, about 1 min)

Teaching note. Objectives and the two-part structure; the big picture from tokens to output.

2 Learning Objectives & Session Structure
3 The Big Picture in One Slide

3.2 2. Why Transformers (slides 4–7, about 2.5 min)

Teaching note. RNN limits (sequential, forgetting, bottleneck); one matrix product computes all queries; Table 1 of the paper; the price of n^2; the analyst resolving "it".

4 Why Recurrent Networks Struggled
5 Running Example: Reading Every Word at the Same Time
6 The Paper's Own Comparison (Vaswani et al., Table 1)
7 The Intuition: An Intelligence Analyst Reading a Report

3.3 3. Tokens & Embeddings (slides 8–12, about 3 min)

Teaching note. Byte-pair tokens; the embedding matrix; directions carry meaning; the worked analogy table (tigress 0.991, queen 0.999, woman 0.995); word space versus context space.

8 Step 1 — Tokenisation: Text to Numbers
9 Step 2 — Embeddings: Placing Words in a Meaning Space
10 Directions Carry Meaning: $man \to woman \approx tiger \to tigress$
11 Worked Analogy: Computing $tiger - man + woman$
12 From Word Space to Context Space

3.4 4. Positional Encoding (slides 13–18, about 4 min)

Teaching note. Order-blindness; requirements for a position code; the sinusoidal formula and frequency table; the $d_{\text{model}} = 4$ example and $E + P$; the heat-map; relative position as rotation (check: $(\sin 5, \cos 5)$); how positions change the attention weights.

13 Step 3 — Why Attention Needs Position Information
14 The Sinusoidal Positional Encoding
15 Running Example: Positional Encoding for "enemy tank advances"
16 The Full Encoding as a Picture: 50 Positions $\times$ 64 Dimensions
17 Why Sinusoids? Relative Position Is a Rotation
18 Running Example: Positions Change What Attention Sees

3.5 5. Query, Key, Value (slides 19–21, about 2 min)

Teaching note. The intelligence-archive analogy; $Q = XW^Q$, $K = XW^K$, $V = XW^V$ and their shapes; Figure 2 (left) read bottom to top.

19 Query, Key, Value: The Intelligence-Archive Analogy
20 Where Do Q, K and V Come From?
21 Scaled Dot-Product Attention (Vaswani et al., Figure 2, left)

3.6 6. Worked Example (slides 22–28, about 4.5 min)

Teaching note. The worked example: Q, K, V; the scores S and $S/\sqrt{2}$; the softmax of row enemy, $(0.0454,\ 0.7679,\ 0.1867)$; the output $Z = AV$, with row advances equal to $(0.5641,\ 1.5760)$; why divide by $\sqrt{d_k}$; the causal mask.

22 Worked Example: Set-Up ("enemy tank advances")
23 Worked Example — Step 1: Queries, Keys and Values
24 Worked Example — Step 2: Scores and Scaling
25 Worked Example — Step 3: Softmax → Attention Weights
26 Worked Example — Step 4: Blend the Values, $Z = AV$
27 Why Divide by $\sqrt{d_k}$?
28 Masked (Causal) Self-Attention: No Peeking at the Future

3.7 7. Multi-Head Attention (slides 29–32, about 2.5 min)

Teaching note. Figure 2 (right); head 2 with $W_2^Q = W_2^K$; concatenation and W^O; what heads learn and why heat-maps are hypotheses.

29 Multi-Head Attention (Vaswani et al., Figure 2, right)
30 Running Example: A Second Head Asks a Different Question
31 Running Example: Concatenate the Heads and Project with W^O
32 What Different Heads Learn in Real Models (Illustrative)

3.8 8. The Paper's Architecture (slides 33–43, about 7.5 min)

Teaching note. Figure 1 block by block: following one English–German sentence with shapes; embeddings and positions; the encoder layer and FFN; Add & Norm worked on the running example (enemy row $(-0.998, 1.094, -0.998, 0.901)$); shifted-right decoder input and masking; cross-attention; Linear and softmax; three uses of attention; settings and results; the three model families.

33 The Transformer of *Attention Is All You Need* (Figure 1)
34 Following One Sentence Through Figure 1
35 **(1) (2)** Input Embedding and Positional Encoding
36 **(3)** The Encoder Layer
37 Running Example: Add & Norm After Multi-Head Attention
38 **(4)** Decoder Input: "Outputs (Shifted Right)" and Masked Attention
39 **(5)** Encoder–Decoder (Cross) Attention
40 **(6)** Linear + Softmax: Output Probabilities
41 Three Uses of Attention in One Model
42 The Paper's Settings and Results
43 Three Families of Transformer Models

3.9 9. Generation & Training (slides 44–45, about 1 min)

Teaching note. Next-token generation and temperature; pre-training loss ($-\ln 0.60 = 0.51$); backpropagation through the whole stack; fine-tuning, instruction tuning, in-context learning.

44 How a GPT-Style Model Writes: Next-Token Prediction
45 How Transformers Are Trained

3.10 10. Defence Applications (slides 46–47, about 1 min)

Teaching note. Text and intelligence, Vision Transformers, sensor fusion, wargaming; limits to brief commanders; misconceptions.

46 Transformers in Defence and Wargaming
47 Common Misconceptions

3.11 11. Summary (slides 48–49, about 1 min)

Teaching note. Summary and quick check; look-ahead to reinforcement learning, where the DQN is a neural network trained by backpropagation.

48 Summary and Quick Check
49 Looking Ahead — Lecture 10: Reinforcement Learning Part 1 — Q-Learning & MDPs

4 Key Messages

Students should leave the room able to state these in their own words:

Key point

1. Self-attention links every token to every other in one step and trains in parallel; the price is n^2 cost.
2. Sinusoidal positions are bounded, unique and turn a shift into a fixed rotation, so relative position is easy to learn.
3. $\text{softmax}(QK^{\mathsf{T}}/\sqrt{d_k})V$: match, scale, normalise, blend; the scaling keeps softmax out of saturation.
4. Multi-head attention asks several questions at the cost of one; Add & Norm keeps deep stacks trainable; the FFN processes each token.
5. Encoder-only, decoder-only and encoder–decoder models share one block and differ in masking, wiring and training objective.

5 Facilitation Tips

- Run the deck as two sessions where possible: Part 1 to the end of the worked example, Part 2 from multi-head attention.
- Use the Attention Workbench in Demo 2 to let students change one entry of W^K and predict which weight moves before revealing it.
- Keep Figure 1 projected while discussing the three model families: point to the parts each family uses.

6 Anticipated Questions and Model Answers

Check your understanding — anticipated question

Q: Why sines and cosines rather than learned positions?

A: The paper found nearly identical results with learned positions and kept sinusoids because they may extrapolate to longer sequences; many later models learn them.

Check your understanding — anticipated question

Q: Why does the decoder need both masked self-attention and cross-attention?

A: Masked self-attention lets it use what it has written so far without seeing the future; cross-attention lets it consult the source sentence.

Check your understanding — anticipated question

Q: How many parameters are in one attention layer of the base model?

A: $4 \times 512^2 + 4 \times 512 = 1{,}050{,}624$, independent of the number of heads.

7 Common Misconceptions

Misconception	Correction
"Positional encoding is concatenated to the embedding."	It is added element by element.
"Q, K and V are three different inputs."	In self-attention they are three projections of the same X.
"Softmax picks the output word."	It gives a distribution; greedy decoding, beam search or sampling picks.

8 In-Class Activities and Discussion Prompts

- **Shape bookkeeping race:** teams write the shape of every tensor in Figure 1 for a 10-word source and 12-word target with the base model; first correct team wins.
- **Family choice:** for six defence text tasks, teams choose encoder-only, decoder-only or encoder–decoder and justify in one sentence each.

9 Run It Live: Studio Demonstrations

Project `Lecture9_Attention_Workbench.html` (it is a single HTML file and runs offline in any browser; every result updates as an input changes). Each demonstration below follows the same rhythm: **set** the inputs, **ask** the class to commit to a prediction (a show of hands or a one-word answer), then **reveal** the result and **land** the point. The numbers are exactly what the page shows for the settings given; simulations use fixed random seeds, so they repeat every time. The 45-minute plan has little slack: choose the one demonstration marked *first choice* for class, and use the others in a tutorial, a second session, or as homework with the drill cards at the bottom of the page.

Demo 1: Positions as a rotation (about 2 min, with §4 Positional Encoding)

Set. Demo 1: $d = 64$, position 1, offset 5.

Ask. "Will the dot product of the codes for positions 1 and 6 match that for positions 30 and 35?"

Reveal. Both **23.504**: each pair contributes $\cos(\omega_i k)$, which depends only on k.

Land the point. A shift is a fixed rotation, so relative position is easy to learn; every value stays in $[-1, 1]$.

Demo 2: Attention on enemy tank advances (about 4 min, with §6 Worked Example) — *first choice*

Set. Demo 2 at the lecture values. Then tick causal mask; then untick divide by $\sqrt{d_k}$.

Ask. "With the mask, what can enemy attend to? Without scaling, will the rows get sharper or flatter?"

Reveal. Unmasked: enemy→tank 76.8%, tank→enemy 57.6%, advances→tank 57.6%, as on the slides. Masked: enemy→enemy 100%, tank→enemy 80.4%, advances unchanged. Unscaled: 86.7%, 66.5%, 66.5%: sharper.

Land the point. Every number on the slides can be reproduced and varied; change one entry of W^K and predict before you look.

Demo 3: Two heads, then Add & Norm (about 2 min, with §8 The Paper's Architecture)

Set. Demo 3 at the lecture values (head 2 with $W_2^Q = W_2^K$).

Ask. "After layer normalisation, what are the mean and spread of each row?"

Reveal. Mean 0 and standard deviation 1; the enemy row is $(-0.998, 1.094, -0.998, 0.901)$ from $\mu = 1.5113$, $\sigma = 0.2344$.

Land the point. Concatenation restores width d so the residual can be added; normalisation keeps deep stacks stable.

Demo 4: The price of context (about 2 min, with §10 Defence Applications)

Set. Demo 4: $n = 4{,}096$, $d = 512$, 16 heads, 2 bytes.

Ask. "How much memory do one layer's attention scores need?"

Reveal. 512 MiB per layer; attention is 57.1% of the multiply-adds, and dominates beyond $n = 6d = 3{,}072$.

Land the point. Long contexts are expensive: chunk, retrieve or use memory-efficient attention.

10 Quiz Administration and Answer Key

Read aloud to students

"You will now take a 15-question quiz. Each question has four options and exactly one correct answer. You have 40 seconds per question; the quiz advances automatically when time runs out. A correct answer scores +1, a wrong answer −0.25, and an unanswered question 0. The maximum is 15 and the minimum is −3.75. Because of the negative marking, guessing blindly among four options gains nothing on average — answer when you can eliminate options. Your score and a full review appear at the end."

Delivery. Open `Lecture9_Quiz.html` on each student device (or project it and collect answers on paper). The page enforces the timer and computes the score with negative marking. Release `Lecture9_Answers_Assignment.html` only after all students have finished.

Answer key

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
C	A	B	D	A	C	B	A	D	B	C	A	D	A	C

Interpreting scores. 12 or more: secure command of the lecture. 9–11.75: good, review missed items. Below 9: revisit the reference text before the assignment. If more than half the class misses a question, revisit that point in the next lecture's opening Q&A.

Questions, options and rationale

Q1. Per Vaswani et al.'s Table 1, the maximum path length between two tokens is $O(1)$ for self-attention and ____ for a recurrent layer.

A. $O(\log n)$
B. $O(n^2)$
C. $O(n)$ ✓
D. $O(1)$

Rationale: An RNN passes information one step at a time; attention links every pair directly.

Q2. The price of self-attention's direct links is a per-layer cost that grows as:

A. $n^2 d$ in the sequence length n ✓
B. $\log n$
C. d^3
D. It does not grow with n

Rationale: Every token scores every token: doubling the context quadruples the score matrix.

Q3. In the toy space, tiger − man + woman = $(-1.00, 0.05, 0.95, 0.00)$. Its nearest word by cosine (inputs excluded) is:

A. lioness
B. tigress ✓
C. female
D. queen

Rationale: Cosines: tigress 0.991, lioness 0.967, female 0.789.

Q4. The sinusoidal encoding uses frequencies $\omega_i = 10000^{-2i/d_{\text{model}}}$. With $d_{\text{model}} = 4$, the second pair has $\omega_1 =$

A. 0.1
B. 0.0001
C. 1
D. 0.01 ✓

Rationale: $10000^{-2/4} = 10000^{-1/2} = 0.01$.

Q5. For $d_{\text{model}} = 4$, the positional encoding of position 1 is about:

A. $(0.8415, 0.5403, 0.0100, 1.0000)$ ✓
B. $(1, 1, 1, 1)$
C. $(0, 1, 0, 1)$
D. $(0.5403, 0.8415, 1.0000, 0.0100)$

Rationale: $(\sin 1, \cos 1, \sin 0.01, \cos 0.01)$. Position 0 is $(0, 1, 0, 1)$.

Q6. Why does the paper say sinusoids let the model attend by relative position?

A. They are learned
B. They are concatenated to the embedding
C. PE_{pos+k} **is a fixed rotation (linear map) of** PE_{pos} **that depends only on** k ✓
D. They are always positive

Rationale: The angle-addition formulas give a rotation matrix in ωk alone.

Q7. In the worked example, $q_{\text{enemy}} = (2, 0)$ and $k_{\text{tank}} = (2, 0)$. The scaled score with $d_k = 2$ is:

A. 4
B. 2.8284 ✓
C. 1.4142
D. 2

Rationale: $4/\sqrt{2} = 2.8284$.

Q8. Row enemy has scaled scores $(0, 2.8284, 1.4142)$. The attention weight on tank is about:

A. 0.768 ✓
B. 0.333
C. 0.867
D. 0.187

Rationale: $16.9188/(1 + 16.9188 + 4.1133) = 16.9188/22.0321 = 0.7679$.

Q9. If query and key components are independent with mean 0 and variance 1, the dot product $q \cdot k$ has variance:

A. 1
B. $\sqrt{d_k}$

C. $1/d_k$

D. d_k ✓

Rationale: A sum of d_k unit-variance terms; hence the division by $\sqrt{d_k}$.

Q10. With the causal mask, the row for "tank" (scaled scores 1.4142 and 0 on enemy and tank) gets weights:

A. $(0.576, 0.140, 0.284)$

B. $(0.804, 0.196, 0)$ ✓

C. $(0.5, 0.5, 0)$

D. $(1, 0, 0)$

Rationale: The future token is set to $-\infty$; renormalising $(4.113, 1)$ gives 0.804 and 0.196.

Q11. In the base transformer, $h = 8$ heads with $d_k = d_v = 64$ are concatenated into a vector of width:

A. 64

B. 8

C. 512 ✓

D. 2048

Rationale: $8 \times 64 = 512 = d_{\text{model}}$; W^O is 512×512.

Q12. How many parameters (with biases) does one multi-head attention layer of the base model have?

A. $4 \times 512^2 + 4 \times 512 = 1{,}050{,}624$ ✓

B. $8 \times 64 \times 64 = 32{,}768$

C. $2 \times 512 \times 2048 = 2{,}097{,}152$

D. It depends on the number of heads

Rationale: W^Q, W^K, W^V, W^O are each 512×512 in total, independent of h.

Q13. In encoder–decoder (cross) attention, block 5 of Figure 1, no mask is needed because:

A. The decoder never sees the encoder

B. Softmax ignores future words

C. Queries come from the encoder

D. The whole source sentence is known before decoding starts ✓

Rationale: Keys and values come from the encoder's output; only the target side has a "future".

Q14. "Outputs (shifted right)" in Figure 1 means that during training the decoder input is:

A. The correct target sentence moved one place right behind a start token ✓

B. The source sentence reversed

C. Random tokens

D. The encoder output

Rationale: Position t predicts word t while seeing only words before it (teacher forcing).

Q15. A defence team needs to tag units, places and equipment in intercepted text. The most natural model family is:

A. Decoder-only (GPT-style)

B. Encoder–decoder for translation

C. Encoder-only (BERT-style), using bidirectional context ✓

D. A recurrent network

Rationale: Tagging is understanding, not generation; bidirectional context helps each token's label.

11 Assignment: Marking Scheme and Model Solutions

Instructions. Five problems, 100 marks, plus a 20-mark bonus problem (Problem 6). Reference: Lecture 9 reference text.

#	Problem	Marks
1	Sinusoidal positional encoding	20
2	Scaled dot-product attention by hand	20
3	Multi-head shapes and parameter counts	20
4	Masking and cost	20
5	BERT, GPT or encoder–decoder for a defence task	20
6	Problem 6 (bonus): Add & Norm by hand	20
	Total	**100** + 20 bonus

General marking guidance

- Marks for each part are shown in square brackets in the problem statement.
- Award method marks for correct working even when an arithmetic slip carries through; deduct once for the slip, not again for its consequences.
- For interpretive parts, full marks need (i) the correct conclusion, (ii) the reason, and (iii) an operational implication or caveat.
- Answers that state a number without uncertainty, baseline or caveat where one is required lose a mark per part.
- Suggested deadline: one week after the lecture; return marked work before the next quiz on the same topic.

11.1 Problem 1: Sinusoidal positional encoding (20 marks)

Problem statement

Take $d_{\text{model}} = 6$.
(a) Compute the three frequencies $\omega_i = 10000^{-2i/6}$. [3]
(b) Compute PE_0, PE_1, PE_2, PE_3 to four decimals. [8]
(c) Verify that $PE_1 \cdot PE_3 = PE_2 \cdot PE_4$ by showing both equal $\sum_i \cos(2\omega_i)$, and evaluate it. [5]
(d) Why is $p_t = t$ a poor position code, and why might sinusoids extrapolate to longer inputs than learned vectors? [4]

Model solution and marking points

(a) $\omega_0 = 1$, $\omega_1 = 10000^{-1/3} = 0.046416$, $\omega_2 = 10000^{-2/3} = 0.0021544$.
(b) $PE_t = (\sin t, \cos t, \sin \omega_1 t, \cos \omega_1 t, \sin \omega_2 t, \cos \omega_2 t)$: $PE_0 = (0, 1, 0, 1, 0, 1)$; $PE_1 = (0.8415, 0.5403, 0.0464, 0.9989, 0.0022, 1.0000)$; $PE_2 = (0.9093, -0.4161, 0.0927, 0.9957, 0.0043, 1.0000)$; $PE_3 = (0.1411, -0.9900, 0.1388, 0.9903, 0.0065, 1.0000)$.
(c) For each pair, $\sin a \sin b + \cos a \cos b = \cos(a - b)$, so $PE_p \cdot PE_{p+k} = \sum_i \cos(\omega_i k)$, which depends only on $k = 2$: $\cos 2 + \cos 0.0928 + \cos 0.0043 = -0.4161 + 0.9957 + 1.0000 = \mathbf{1.5795}$ for both.
(d) $p_t = t$ is unbounded: at position 1,000 it would swamp the meaning in the embedding,

and its scale depends on length. Sinusoids are bounded in $[-1, 1]$ and defined for any position by the formula, with relative shifts as fixed rotations, whereas learned vectors exist only for positions seen in training.

11.2 Problem 2: Scaled dot-product attention by hand (20 marks)

Problem statement

Three tokens with $X = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$, $W^Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$, $W^K = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$, $W^V = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$, $d_k = 2$.

(a) Compute Q, K and V. [5]
(b) Compute $S = QK^\mathsf{T}$ and $S/\sqrt{2}$. [5]
(c) Compute the attention weights A (four decimals). [6]
(d) Compute $Z = AV$. [4]

Model solution and marking points

(a) $Q = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 2 & 1 \end{pmatrix}$, $K = \begin{pmatrix} 1 & 2 \\ 1 & 1 \\ 2 & 1 \end{pmatrix}$, $V = \begin{pmatrix} 2 & 1 \\ 1 & 2 \\ 3 & 1 \end{pmatrix}$.

(b) $S = \begin{pmatrix} 3 & 2 & 3 \\ 5 & 3 & 4 \\ 4 & 3 & 5 \end{pmatrix}$; $S/\sqrt{2} = \begin{pmatrix} 2.1213 & 1.4142 & 2.1213 \\ 3.5355 & 2.1213 & 2.8284 \\ 2.8284 & 2.1213 & 3.5355 \end{pmatrix}$.

(c) Row 1: exponentials $(8.3421, 4.1133, 8.3421)$, sum 20.7975: $(0.4011, 0.1978, 0.4011)$. Row 2: $(34.3133, 8.3421, 16.9188)$, sum 59.5742: $(0.5760, 0.1400, 0.2840)$. Row 3: $(16.9188, 8.3421, 34.3133)$: $(0.2840, 0.1400, 0.5760)$.

(d) $Z = \begin{pmatrix} 2.2033 & 1.1978 \\ 2.1440 & 1.1400 \\ 2.4359 & 1.1400 \end{pmatrix}$. E.g. row 3: $0.284(2,1)+0.14(1,2)+0.576(3,1) = (2.4359, 1.1400)$.

11.3 Problem 3: Multi-head shapes and parameter counts (20 marks)

Problem statement

BERT-base: $d_{\text{model}} = 768$, $h = 12$, feed-forward width 3,072, input of $n = 128$ tokens.
(a) Give d_k and the shapes of Q_i, K_i, V_i, the score matrix of one head, the concatenation and the output. [6]
(b) How many attention scores are computed per layer? [3]
(c) Count the parameters of the attention sub-layer and of the feed-forward sub-layer. [7]
(d) Why must the output have the same width as the input? [4]

Model solution and marking points

(a) $d_k = 768/12 = 64$. Q_i, K_i, V_i: 128×64; scores per head 128×128; concatenation 128×768; after W^O (768×768): 128×768.
(b) $12 \times 128 \times 128 =$ **196,608**.
(c) Attention: $4 \times 768^2 + 4 \times 768 =$ **2,362,368**. Feed-forward: $2 \times 768 \times 3{,}072 + 3{,}072 + 768 =$ **4,722,432**. The feed-forward layer holds two-thirds of the block.
(d) So the output can be added to the input in the residual connection and so identical blocks can be stacked.

11.4 Problem 4: Masking and cost (20 marks)

Problem statement

(a) Apply the causal mask to the scaled scores of Problem 2 and recompute A and Z. Which row is unchanged? [8]
(b) By what factor does the number of attention scores grow when the context goes from 512 to 2,048 tokens? [4]
(c) A model has 16 heads and stores scores in 2-byte numbers. How much memory do one layer's score matrices need at 4,096 tokens? [4]
(d) Where in Figure 1 is the mask used, and where is it not, and why? [4]

Model solution and marking points

(a) Row 1: $(1, 0, 0)$, $z_1 = v_1 = (2, 1)$. Row 2: softmax of $(3.5355, 2.1213)$: $(0.8044, 0.1956, 0)$, $z_2 = (1.8044, 1.1956)$. Row 3 sees everything and is unchanged: $(0.2840, 0.1400, 0.5760)$, $z_3 = (2.4359, 1.1400)$.
(b) $(2048/512)^2 = \mathbf{16}$ times.
(c) $4{,}096^2 \times 16 \times 2 = 536{,}870{,}912$ bytes = **512** MiB per layer, before any other activations: why long contexts are expensive.
(d) Used in the decoder's masked self-attention (block 4), so position i cannot see later target words. Not used in encoder self-attention (the whole source is known) or in encoder–decoder attention (keys and values are the known source).

11.5 Problem 5: BERT, GPT or encoder–decoder for a defence task (20 marks)

Problem statement

For each task choose a model family and justify it: (a) tagging units, places and equipment in intercepted messages; (b) drafting a SITREP from field notes; (c) translating captured documents. [12]
(d) For task (b), name two failure modes and a safeguard for each. [8]

Model solution and marking points

(a) Encoder-only (BERT-type). Tagging is understanding: each token's label benefits from context on both sides, which the unmasked encoder provides; a small classification head sits on each token's vector.
(b) Decoder-only (GPT-type), or an encoder–decoder. Drafting is generation; a decoder trained to predict the next token, given the notes as a prompt, writes the report one token at a time.
(c) Encoder–decoder, the original transformer's task: the encoder reads the source language bidirectionally; the decoder writes the target, consulting the source through cross-attention.
(d) Hallucination: fluent invented details; safeguard: require each statement to cite its source note and have an officer verify before release. **Prompt injection or adversarial text** in the notes: safeguard: treat inputs as data, restrict output to the report template, and log inputs for audit. (Also acceptable: distribution shift to unfamiliar jargon, with fine-tuning and testing on unit data.)

11.6 Problem 6 (bonus): Add & Norm by hand (20 marks)

Problem statement

(a) A token's input is $x = (2, 0, 1, 1)$ and its sub-layer output is $(0.5, 1.5, -0.5, 0.5)$. Compute $h = x + \text{Sublayer}(x)$, its mean, variance and standard deviation, and $\text{LN}(h)$ with $\gamma = 1$, $\beta = 0$. [8]
(b) Recompute with $\gamma = (1, 1, 2, 2)$ and $\beta = (0, 0, 0.5, 0.5)$. [4]
(c) Repeat (a) for $x = (1, 3, 0, 2)$ and sub-layer output $(1, 1, 0, -2)$. [5]
(d) Why does a transformer normalise each token separately rather than over the batch? [3]

Model solution and marking points

(a) $h = (2.5, 1.5, 0.5, 1.5)$; mean 1.5; deviations $(1, 0, -1, 0)$; variance 0.5; s.d. 0.7071. $\text{LN}(h) = (\mathbf{1.4142},\ \mathbf{0},\ \mathbf{-1.4142},\ \mathbf{0})$.
(b) $\gamma \odot \text{LN}(h) + \beta = (1.4142,\ 0,\ -2.8284 + 0.5,\ 0 + 0.5) = (1.4142,\ 0,\ -2.3284,\ 0.5)$.
(c) $h = (2, 4, 0, 0)$; mean 1.5; deviations $(0.5, 2.5, -1.5, -1.5)$; variance $(0.25 + 6.25 + 2.25 + 2.25)/4 = 2.75$; s.d. 1.6583. $\text{LN}(h) = (0.3015,\ 1.5076,\ -0.9045,\ -0.9045)$.
(d) Sequences vary in length and padding, and at inference a single sequence may be processed alone; per-token statistics do not depend on the other examples in the batch, so training and inference behave identically.